
25

各製品で Claude をどう封じ込めているか

— *How we contain Claude across products* —

公開日	2026-05-25
原題	How we contain Claude across products
著者	Anthropic Engineering Team
原文	https://www.anthropic.com/engineering/how-we-contain-claude
翻訳	Claude (機械翻訳／Anthropic)
編集	2026-05-27

各製品で Claude をどう封じ込めているか

12 か月前であれば、Anthropic 社内サービスを停止させられるだけのアクセス権を Claude に与えるという発想は即座に却下していたはずですが。今日その水準のアクセスは日常的なものとなり、Anthropic の開発者はそれによって生産性を上げています。こうした展開のリスクは 2 つの要素に分解できます。すなわち、失敗の起こりやすさと、起きた場合に与え得る損害の大きさです。安全策とモデル訓練の進展により前者は着実に下がってきましたが、後者——理論上の爆発半径——は能力とアクセスが拡大するにつれてむしろ膨らみ続けます。とはいえ、エージェントがかつて 1 人、あるいはチームを要した仕事をこなせるようになると、展開しない ことのコストが大きくなりすぎ、製品を安全にできる限り、リスクとリターンの天秤は採用側へ大きく傾きます。エンジニアリング上の問いは、どのようにして爆発半径に蓋をするか、ということになります。



自律エージェントが及ぼし得る相対的な損害に——たとえば実行環境への制御を通じて——境界を設けられるとき、高い有用性をもつ能力は展開を後押しします。Claude Mythos Preview は、2026 年 4 月時点で爆発半径が大きすぎると判断され出荷を見送ったモデルの一例です。一方で、同程度の能力を備えたモデルの広範なリリースは、防御側が重要システムを堅牢化し安全策が成熟するにつれ適切になると見ています——もちろん、いかなる場合もある程度のリスクは残ります。モデルの能力は、エージェント展開における総合リスクを左右する重要な因子です。

これを実現する方法は大きく 2 つあります。

1 つ目は、ヒューマン・イン・ザ・ループでエージェントの挙動を監督する方法です。Claude Code はかつて、各ターンでユーザーに許可を求めることで、エージェントが意図しない操作を行うのを防いでいました。理論上はこれで成立しますが、私たちはこのアプローチが万能ではないことを確認しました。テレメトリでは、ユーザーは許可プロンプトのおよそ 93% を承認していました。承認プロンプトを多く目にすればするほど、1 つひ

とつへの注意は薄れ、時間が経つにつれ監督は格段に疎かになります。最近私たちは、この承認疲れを軽減するために、より安全な承認を 自動化 する Claude Code auto mode を構築しました。それでも脆弱性は残ります——確率的な防御である以上、見逃し率はゼロにはなりません。1

爆発半径に蓋をする 2 つ目のアプローチが封じ込め (containment) であり、本稿の主眼でもあります。エージェントが 何をするか を監督するのではなく、サンドボックスや仮想マシン、エグレス制御などを通じてアクセス境界を強制し、エージェントが 何をできるか を監督します。ここに Anthropic Engineering は最も多くの労力を注いできましたし、最も意外なセキュリティ上の失敗の多くもここで起きてきました。

過去 2 年間、私たちは 3 つの主要なエージェント製品を出荷してきました——claude.ai、Claude Code、Claude Cowork です。それぞれが異なる利用者を対象としており、求められる封じ込めアーキテクチャも異なります。本記事では、何が持ちこたえ、何が壊れ、エージェントセキュリティについて道中で学んだことを共有します。

3 種類のリスクと、防御の 3 つの構成要素

エージェントに対するセキュリティリスクは、おおむね次の 3 つのカテゴリーに分類できます。

ユーザーによる誤用: ユーザーが——悪意からであれ、不注意からであれ——エージェントに有害な行為を指示します。鬱陶しいチェックを回避するよう頼むものから、意味のわからない破壊的コマンドを実行するもの、明示的に害を意図したものまで含まれます。

モデルの誤挙動: 誰も頼んでいないのにエージェントが有害な行動を取ります。私たちのモデルが改善されるにつれ、ほとんどの行動評価ではアラインメントが進んでいましたが、これはリスクが必ずしも小さくなることを意味しません。能力の低いモデルは状況を読み違え、明らかな誤りを犯しやすい一方、能力の高いモデルはミスが少ない代わりに、誰も明文化を思いつかなかった制約を回り込みながら、ゴールへの予想外の経路を見つけるのも上手くなります。

Anthropic では、Claude モデルがタスクを完遂しようとして「親切心で」サンドボックスから抜け出す、コーディングテストの答えを探そうと git 履歴を漁る、走っている ベンチマークを自発的に同定して答えキーを復号する、といった事例を目撃してきました。各モデルは新たな能力を伴って登場し、それが時に予期せぬ形で発揮されます。

外部からの攻撃者: ツール、ファイル、ネットワークアクセスといった外部ベクトルを通じてエージェントが攻撃されます。このカテゴリーには、プロンプトインジェクションだけでなく、エージェントのランタイム、オーケストレーション層、プロキシに対する従来型の攻撃も含まれます。

封じ込めと防御の仕組みを構築する際、私たちは主に次の 3 つの構成要素に防御を適用します。

エージェントが動作する環境。 プロセスサンドボックス、VM、ファイルシステム境界、エグレス制御によって、エージェントがどこでどう動けるかを制約します。狙いは、エージェントが届く範囲に対し硬い境界を設けることです。たとえばクレデンシャルがサンドボックスに入らなければ、原因がユーザーであれ、「創造的」な経路を見つけたモデルであれ、攻撃者であれ、漏出させようがありません。

境界が厳密になれば、その分監督を緩めることもできます。Claude Code の [リファレンス devcontainer](#) は、まさにエージェントが操作ごとの承認なしに無人で動けるようにするために存在しています。

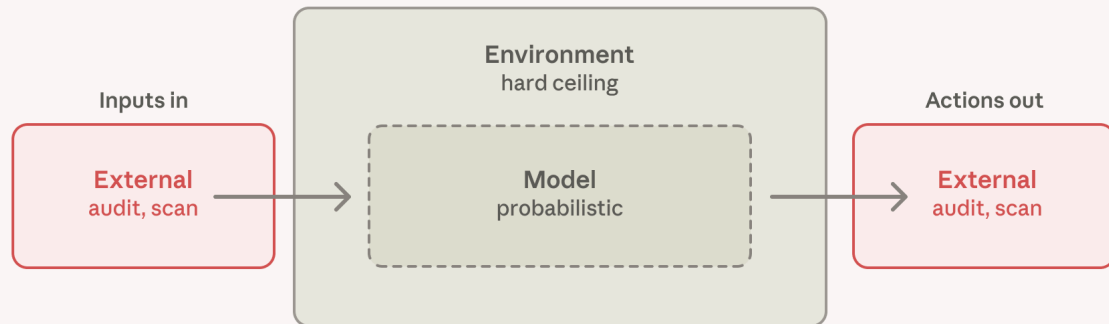
エージェントが参照するモデル。 ここで使う仕組みには、システムプロンプト、分類器、プローブ、訓練の改変があります。モデルは確率的なので、これらはあくまでエージェントが *何をしがちか* を形作るだけであり、理論上 *何ができるか* を変えるわけではありません。

これらの防御は強力です。プロンプトインジェクションに対する耐性をテストする Gray Swan の Agent Red Teaming ベンチマークで、[Claude Opus 4.7](#) は単発攻撃の成功率を約 0.1%、100 回の適応的攻撃後でも 5〜6% に抑えています。Claude Code auto mode は、行き過ぎた挙動のおよそ 83% を [実行前に](#) 検知します。とはいえ、業界最高水準の防御をもってしても、モデル層の保護が 100% 効くことはあり得ません。だからこそ単独で立たせるわけにはいきません。

エージェントが届く外部コンテンツ。 MCP サーバー、サードパーティのプラグイン、ウェブ検索ツールはいずれも、あなたの制御外のソースからエージェントの文脈にコンテンツを流し込みます。監査されたコネクタは、監査されたデータと同義ではありません——たとえば GitHub コネクタは、マルウェアチェックを通過しながら、汚染された README をそのままモデルの文脈に取り込んでしまう可能性があります。ツール権限をきめ細かく絞れば、爆発半径を抑えるのに役立ちます。たとえば DB に読み取り専用アクセスしか持たないエージェントは、本番に書き込めるエージェントよりはるかに広い範囲に展開できます。

防御は重ね、補完し合うべきです。環境防御が使えないときは、モデル層が穴を埋めなければなりません（まさにこのために Claude Code の [auto mode](#) は設計されています）。ローカル領域では、環境防御とモデル防御が悪意あるツール出力に対する盾になります。同時に、ツールの能力やアクセスを制限することで、より上流に防御を追加できます。

Components to defending AI agents



防御すべき3つの構成要素: モデル、それが動作する環境、エージェントが届ける外部コンテンツ。

エージェントを封じ込めるパターン

ここからは環境層に焦点を当て、3つの隔離パターンと、それを各 Claude プラットフォーム——claude.ai、Claude Code、Cowork——向けにどう仕立てているかを説明します。各設計には、エージェントに求める能力と、ユーザーに求める介入度のバランスを見極めながら、段階的にたどり着きました。

パターン 1: エフェメラルなコンテナ (claude.ai のコード実行)

claude.ai はチャットインターフェイスとして広く知られていますが、コードの記述・実行、ファイル生成、コネクタ呼び出しも行います。Claude が claude.ai 内でコードを実行する際は、隔離されたインフラ上の [gVisor](#) コンテナで実行されます。エージェントは完全にサーバーサイドで動作し、ローカルマシンではコードを一切実行しません。ファイルシステムはエフェメラル (セッションごと) です。爆発半径は最小限ですが、その分 Claude にできることの上限も限定的で、永続的なワークスペースもユーザーのファイルシステムへのアクセスもありません。

これにより [claude.ai](#) は、より伝統的な脅威モデルの対象でもあります。私たちが守るのはエージェントからユーザーのマシンではなく、私たち自身のインフラと、テナント同士の隔離です。[claude.ai](#) のローンチ前作業の多くは、ネットワーク構成、内部サービスの認証、オーケストレーションといった伝統的なセキュリティ業務に費やされました。

その作業はセキュリティで最も古い教訓を改めて裏付けました——最弱の層は、自分で作った層だということです。gVisor と [seccomp](#) は、エージェント型 AI が登場するよりもずっと長い間、潤沢なリソースを持つ攻撃者に対して堅牢化されてきたので、レビューの労力はその周辺に私たちが新たに構築した部分に集中させました。後ほど触れますが、私たちのカスタムプロキシは、最も影響の大きかったインシデントで壊れた部分でもあります。

パターン 2: ヒューマン・イン・ザ・ループ型サンドボックス (Claude Code)

Claude Code はユーザーのマシン上で動作し、ファイルシステム、シェル、ネットワークにアクセスします。これがないとコーディングエージェントは大した役に立たないので、この権限を安全に与える方法を見つけることが不可欠です。

ひとつの方法は、ヒューマン・イン・ザ・ループに頼ることです。これが現実的な解になるのは Claude Code に限ります。平均的なユーザーは開発者で、コーディング環境に慣れているからです。彼らは `bash` を読めますし、`rm -rf` の意味を理解していますし、信頼できないソースから `npm install` を週に何度も実行しています。つまり、「これを許可しますか？」のダイアログが出たとき、エージェントが何をしようとしていて、どんなリスクがあるかを正しく評価する知見をユーザーが持ち合わせている可能性が高いということです。これを踏まえて、Claude Code は最もシンプルな防御で出発しました——読み取りは許可、書き込み・bash・ネットワークアクセスは承認制です。

しかし前述の通り、承認疲れは [数週間うちに表面化しました](#)。皮肉なことに、本来監督を強化するために設計された機能が逆効果を生じさせ得る——一部のユーザーは単に注意を払うのを止めるかもしれない——という構図になっていました。不用意な承認を緩和する第一歩として、私たちは OS レベルのサンドボックス (macOS では Seatbelt、Linux では bubblewrap) を出荷し、境界を堅くしました。読み取りは許可、ワークスペース内の書き込みは許可、しかしネットワークはデフォルトで拒否、というものです。サンドボックス内では、エージェントはほぼ中断なしに動作します。結果として許可プロンプトは 84% 減少し、境界が監査可能なように [このランタイムをオープンソース化](#) しました。

私たちの [匿名化された利用データ](#) によると、経験豊富なユーザーは新規ユーザーのおよそ 2 倍の頻度で自動承認する一方、エージェントの実行を途中で割り込ませる頻度も高いことが分かりました。経験豊富なユーザーは個々のステップにゲートをかけるのではなく、エージェントが脱線したときだけ監督する傾向があるのです。これは人がエージェントとの働き方として自然にたどり着く進化かもしれませんが、ここにも盲点があります。そもそも脱線に気付くだけの技術力と注意力をユーザーが持っている必要があるからです。モデルの能力が向上し、エージェントがより野心的な `bash` を書くようになると、脱線に気付くこと自体が難しくなります。さらにユーザーがマルチエージェントシステムへ移行すれば、このやり方が有効な監督戦略になる可能性はさらに低下します。

見落とししたリスク:信頼ダイアログより前のすべて

2025 年中ごろから 2026 年 1 月にかけて、私たちは責任ある開示プログラムを通じて Claude Code の脆弱性報告を受け取りました。そのうち 3 件は、ユーザーが何にも同意する前に実行されるコードを突くものでした。最も直接的なケースを考えれば、これがどう成立するか分かります。開発者がプルリクエストをレビューしようとリポジトリをクローンする。そのリポジトリには、フックを定義した `.claude/settings.json` が含まれている。Claude Code は標準の「このフォルダを信頼しますか？」プロンプトを表示する前に起動時にプロジェクト設定を読むため、攻撃者が用意してコミットしておいたフックが自動的に実行されてしまいます。残りのケースも構造的によく似ていて、まだ信頼されていないディレクトリからの入力、信頼境界が確立される前にパースされていました。

各ケースでの修正は同じ形をしていました。プロジェクトローカルな設定のパースと実行を、ユーザーが信頼プロンプトを受け入れた後まで遅延させる、というものです。同様のものを構築しているなら、プロジェクトを開く処理、設定の読み込み、localhost のリスナーは、インターネットから到着する受信リクエストと同じ扱いにすべきです。ローカル感があり、ユーザーが同意するより先に届くからといって、暗黙に信頼してよい理由にはなりません。

見落とししたリスク:注入経路としてのユーザー

2026 年 2 月、社内で管理下に行われたレッドチーム演習において、ある研究者が従業員をフィッシングし、悪意あるプロンプトと共に Claude Code を起動させることに成功しました。フィッシングメールはありふれた共同作業のような見た目——「これ走らせてもらえる？」と、すぐ貼れる形のプロンプトが添えられていました——で、プロンプト自体もごく普通のタスク指示のように読めました。しかしセットアップ手順のどこかに、`~/aws/credentials` を読み、内容をエンコードし、外部エンドポイントに POST してほしいと、さりげなく Claude に頼む文が紛れていました。同じプロンプトを 25 回繰り返したところ、Claude は 24 回その流出を完了しました。

これは *直接的な* プロンプトインジェクションです。攻撃者の指示はツール出力や取得コンテンツ経由ではなく、ユーザーを通じて到来しました。私たちのモデル層の防御はユーザー意図を起点に判断しているため、ユーザー自身が指示を打ち込んでいる状況では、分類器が引っ掛かる異常が存在しません。同じスクリプトを手渡された人間の業務委託者であれば、同じことをしていたでしょう。

この状況で持ちこたえる唯一の防御は環境層、特に意図に関係なく POST をブロックするエグレス制御と、そもそも `~/aws` を手の届かないところに置くファイルシステム境界です。

(このプロンプトを社内 Slack で議論用に共有したところ、社内エージェントの中には Slack を読むものもあると指摘されました。ペイロードが今や常時漂う状態になったのです。私たちはスレッドに [カナリア文字列](#)を追加し、何かが拾った場合に気付けるようにしました。エージェントがあらゆるものを読み込む世界では、調査ツール自体も攻撃面となります。)

パターン 3: ローカル VM (Claude Cowork)

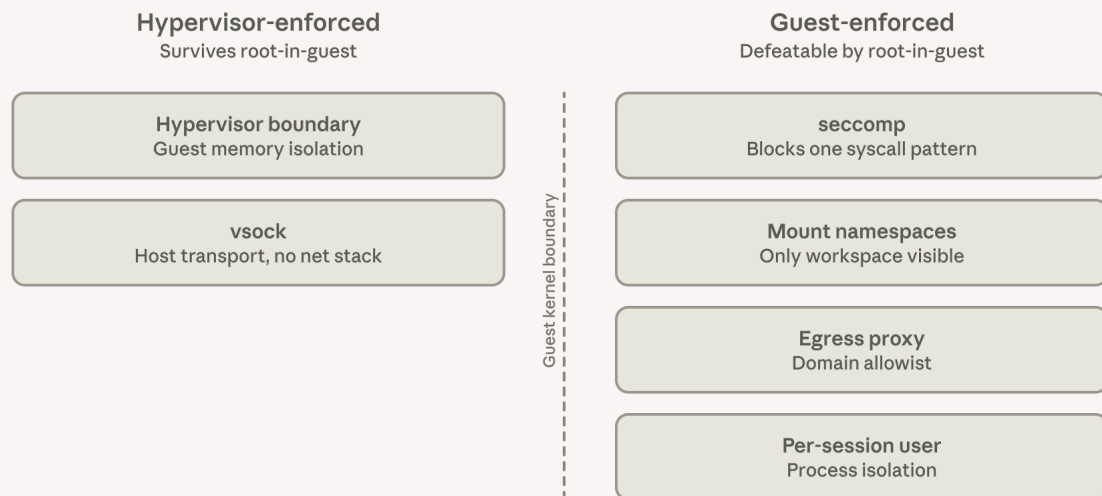
Claude Cowork はユーザーのデスクトップ上で動作し、ユーザーが選んだワークスペースフォルダにアクセスします。このプラットフォームはソフトウェアエンジニアリングではなく一般的なナレッジワークのために作られているので、平均的なユーザーが bash に堪能である可能性ははるかに低くなります。

その結果、ヒューマン・イン・ザ・ループ型サンドボックス戦略はそのままでは通用しないでしょう。非エンジニアのナレッジワーカーに、`find . -name "*.tmp" -exec rm {} \;` のような bash 呪文を判断させるべきではありません。例外を承認するのに、典型的なユーザーが持たない専門知識が要るのであれば、管理者は絶対的かつ常時オンの境界を設定すべきです。

これを実現するため、Claude Cowork の最初のバージョンは、プラットフォーム提供のハイパーバイザー (macOS では Apple の Virtualization フレームワーク、Windows では HCS) を使った完全な仮想マシン内で動作していました。VM は独自の Linux カーネル、独自のファイルシステム、独自のプロセステーブルを持ちます。ユーザーが選んだワークスペースと `.claude` フォルダはマウントされ、それ以外のホスト上の要素は見えません。クレデンシャルはホストのキーチェーンに留まり、ゲストマシンに入ることはありません。この設計は、Claude がいつかどこかで誤ったアラインメントの挙動を取る可能性に備えています。侵害された Claude はワークスペースフォルダ内のものを破壊する余地はあるものの、アーキテクチャはそれが 唯一 届く場所であるよう設計されており (ユーザーがコネクタを追加するまでは)、そこに何をマウントするかをユーザーが制御します。

オリジナルのアーキテクチャ——フル VM モードと私たちが呼ぶもの——では、エージェントループ自体がゲスト内で実行されており、Claude はサンドボックス化されていることを自覚しないまま、ごく普通の Linux ユーザーとして動いていました。Claude Code と比べてみてください。あちらでは特権プロセスがサンドボックスの外に居座り、コマンドごとに強制するかを決めます。説得力のあるインジェクションプロンプトや、疲れた末の承認クリックひとつで、そのプロセスはサンドボックス外の何かを実行し得ます。一方こちらでは、脱出ハッチの鍵を握る外側プロセスが存在せず、例外を許可する権限を持つコンポーネントも存在しません。

Claude Cowork's isolation mechanisms

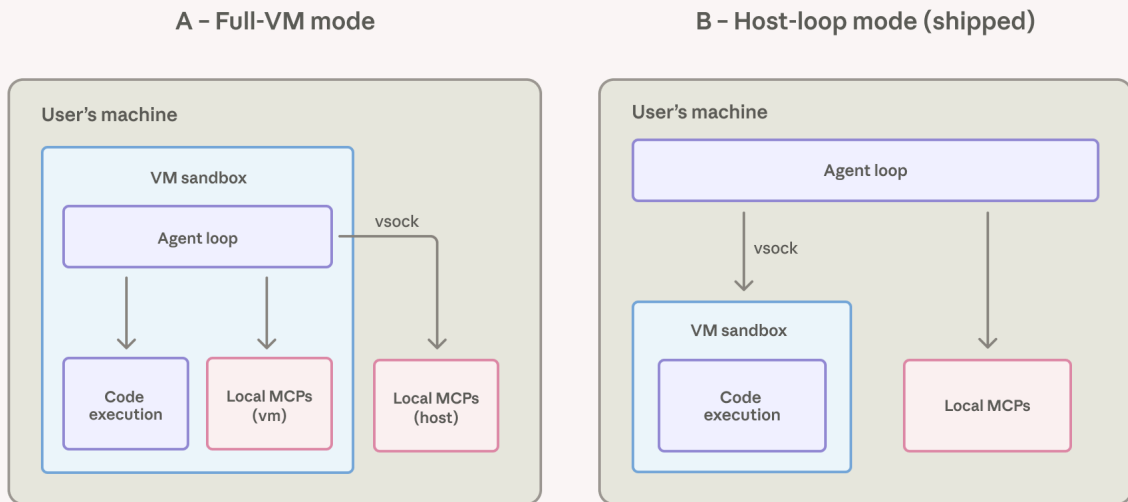


Claude Cowork の VM における主な 6 つの隔離メカニズム。うち 2 つはゲストカーネルの外で強制されており、エージェントが VM 内で root 権限を取っても生き残ります。残り 4 つはゲスト側で強制され、外側の層が残りを担うので意図的に最小限に保たれています。

しかしすぐに、エージェント全体をフル VM モードで動かすことには実用上の問題があると分かりました。VM 起動中のどんな失敗も、Cowork を使えなくしてしまうのです。エージェントループを VM の 外に移し、コード実行は中に残すことで、エラーで凍り付くのではなく、Claude がユーザーに応答して問題のデバッグを手伝えるようになりました。この変更によるセキュリティへの影響は最小限です——VM はエージェントが実行するコードに対して、依然としてファイルシステムとネットワークの制御を強制するからです。

これとは別に、ローカル MCP サーバーも VM の外に移しました。VM 内で動かすと監査が難しくなり、VM 更新時に依存関係が脆くなる問題が出て、データベースなどローカルプロセスとの対話を必要とする MCP もサポートできませんでした——そうしたサーバーはどのみちホスト上で動かす必要がありました。この変更により、Claude Cowork はローカル MCP サーバーが Claude Desktop で動作するのと同じ流儀になります——それらをユーザーがインストールを選ぶ任意のソフトウェアと同様に扱い、どのローカル MCP を有効にする（あるいは有効にしない）かを管理者の判断に委ねる、というものです。リモート MCP サーバーはユーザーのマシン上で動かないため、影響を受けません。

Comparison: Full-VM vs host-loop



エージェントループを VM 内に置く構成では、VM 内のどんな失敗も Cowork を使えなくしました。ホストモードはより信頼性が高く、VM がクラッシュしてもエージェントが応答できる一方、コード実行を隔離することで重要なセキュリティ保証を依然として提供します。

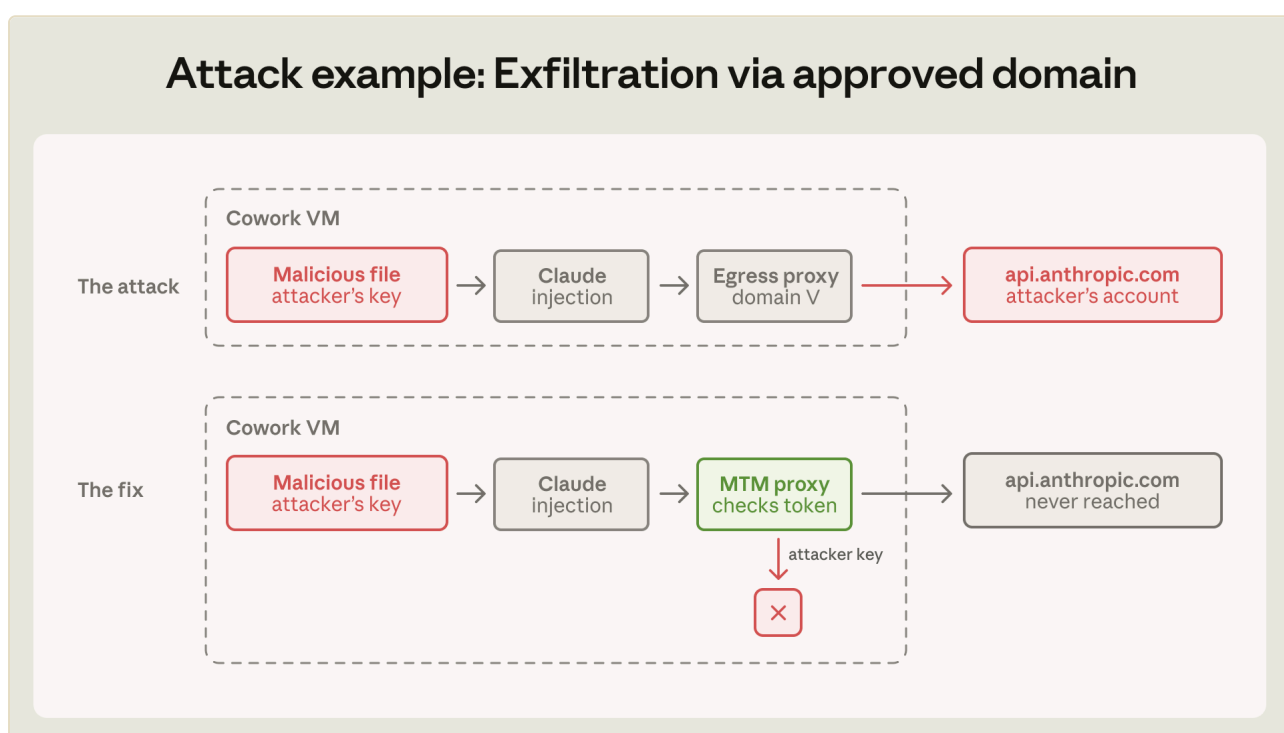
ファイルシステム制御も、もう 1 つの重要なアーキテクチャ上の選択でした。Claude が有用であるためにはホスト上の いくつかのファイルにアクセスする必要がありますが、爆発半径を最小化し、ローカルファイルアクセスについてユーザーに透明性を提供したいと考えました。そこで、ファイルマウントのモードを複数提供することで、リスクを細やかに制御できることが分かりました。Claude Cowork は read-only、read-write、read-write-no-delete を提供しています。ここで 1 つ落とし穴になり得るのは、シンボリックリンクの解決をパス検証の後ではなく 前に行わなければならないという点です。さもないと、許可されたフォルダ内のシンボリックリンクが外を指してしまい、抜け穴になり得ます。エンタープライズ顧客向けには、管理者が MDM 設定でマウントパスの許可リストを管理できるようにしています。

見落とししたリスク: 許可されたドメインを経由した流出

許可されたドメインを経由した流出のはっきりした例は、サードパーティからの開示で判明しました。Claude Cowork のエグレス許可リストは `api.anthropic.com` への通信を正しく通していました——製品は私たち自身の API を呼ばずには機能しないからです。今回のケースでは、ユーザーのマウントワークスペース内に置かれた悪意あるファイルが、攻撃者が制御する API キーと隠された指示を運び込みました。Claude はその指示に従い、ワークスペース内の他のファイルを読み、攻撃者のキーを使って Anthropic の Files API を呼び出しました。エグレスプロキシは宛先をチェックし、`api.anthropic.com` を見て、通しました。ファイルは攻撃者の Anthropic アカウントにアップロードされました。サンドボックスは完璧に機能していたのに、データは流出したのです。

それまで私たちは許可リストを宛先フィルタ、つまり Claude に「これらのドメインなら話して大丈夫だよ」と伝えるものとして概念化していました。しかし、これはむしろ能力付与 (capability grant) として捉えるべきものかもしれません。許可リスト上のドメインから到達できるあらゆる機能は、いまや攻撃面です。api.anthropic.com を許可することは、任意の Anthropic アカウントへのファイルアップロードを許可することを意味していました。

修正には、VM 内に置いた防御的な man-in-the-middle プロキシを使い、API への通信を傍受しました。VM 自身がプロビジョニングされたセッショントークンを携える要求だけを通します。攻撃者が埋め込んだキーはプロキシで拒否されます。サーバーサイドフェッチを有効化するヘッダもブロックします。プロキシをサーバー側ではなく VM 内に置くのは、出自を知っているのが VM だけだからです。サーバーから見ると、Cowork のリクエストは他の API クライアントと見分けがつきません。



上:api.anthropic.com への通信が通り、結果として流出が起きる。下:API への通信を傍受する man-in-the-middle プロキシによる修正。

これもまた、自分で作るソフトウェアこそ最弱になりがち、という原則の二度目の表れです。ハイパーバイザー、seccomp、私たちの製品横断で使っている gVisor は信頼に足るものでした。失敗したのは、私たち独自の許可リストプロキシでした。

見落としたリスク:VM 隔離はエンドポイント検知ソフトも締め出してしまった

Claude Cowork を評価する際、エンタープライズのセキュリティチームから「なぜうちの EDR が中を見られないのか?」と尋ねられました。答えは、Claude を封じ込めている隔離が、ホストベースのエンドポイント検知&対応 (EDR) もまた締め出してしまっているから、というものでした。EDR の視点では、Claude Cowork は不透

明なハイパーバイザープロセスです。ゲストの内側を覗くことはできません。

隔離は可視性を下げます。そして不透明性は、コンプライアンス姿勢がエンドポイント可視性に依存しているチームにとって問題になります。私たちの現状の緩和策はプル型の [OTLP](#) エクスポートで、管理者が事後にイベントログを取得できるようにすることですが、これはライブ監視と同じものではありません。同様のものを構築しているなら、この会話に備える時間を早めに見込んでおくといでしょう。

環境	エフェメラルコンテナ (claude.ai)	HITL サンドボックス (Claude Code)	密閉 VM (Claude Cowork)
コスト:隔離 オーバーヘッド	コンテナの起動	低レイテンシのネイティブサンドボックス	フル VM のブート
コスト:ユーザー依存度	N/A	bash を読める必要あり	N/A
リスク:爆発半径	サーバーサイドコンテナ (gVisor とホストインフラ境界で防御)	ローカルワークスペース	マウントされたワークスペース (vsock とハイパーバイザー境界で防御)

エージェントが読むものを信頼すること

エンタープライズ顧客はよく、MCP 接続をどう安全にするかと尋ねます。良い問いではありますが、本来は MCP に限らずもっと広く考えるべきです。エージェントに与えるあらゆる外部リソースは、同時に 2 つのリスクを示しています——伝統的なサプライチェーンの意味でのコード実行リスクと、プロンプトインジェクションのベクトルです。従来の依存関係監査（バージョンの固定、署名の検証、ソースのレビュー）は前者には効きますが、後者を見落としします。

リモートかローカルかは、見た目以上に重要です。 ローカルインストールされたツールは監査可能です。コードを読み、バージョンを固定し、勝手に変わらないことを確認できます。一方、リモートのツール——ホスト型 MCP サーバーやクラウドコネクタ——は、あなたが承認した後の任意の時点で挙動を変え得ます。インストール時の信頼判断はもはや有効ではなくなるかもしれません。私たちの [コネクタディレクトリ](#) は継続的なレビューでこの問題に対処していますが、それ以外のものは信頼できないものとして扱うべきです。まずはダミーデータで、悪意あるツールの爆発半径が封じ込められる環境で走らせてください。

ツールが信頼できる場合でも、ツール出力は攻撃面です。先に挙げた GitHub README の例はまさにこの場合です。ウェブページに適用する入力スキャンと同じ厳密さで、ネットワーク対応ツールの結果にもスキャンを適用する必要があります。レイテンシが増え、完璧な防御にはなりません。私たちはライブ検査を優先します。汚染されたツールの戻り値がいったんエージェントを操ってデータ流出に至らせると、ログには成功した認可済みの API 呼び出ししか残りません。事後に手繰れるシグナルは存在しないのです。

Claude Code と Claude Cowork では、ツール呼び出しはネットワークとファイルのポリシーを強制するプロキシを通り、戻り値がモデルの文脈に入る前に検査できます。検査を行う分類器は小さくて高速なモデルでよく、推論を担うモデルである必要はありません。

これからの展望

モデルと製品は急速に進歩しています。それに伴い、リスクは形を変え進化し、私たちの緩和策もそれに追従しなければなりません。

永続的なメモリ汚染。セッションをまたいで残るエージェント文脈の比率は増え続けています——製品メモリ、CLAUDE.md ファイル、マウントされたワークスペース、スケジュール実行や長時間稼働するエージェントの状態ディレクトリなどが含まれます。これらのどれかにインジェクションが着地すれば、エージェントが起動するたびに再ロードされます。エージェントの状態がセッションを超えて生き残るほど、古典的なポストエクスプロイト的な意味での、新たな永続化メカニズムに脅かされます。セッション開始時の良い分類器の常設化が、より一般的になっていく必要があります。

マルチエージェントにおける信頼の昇格。一方では、サブエージェントは信頼できない内容を隔離し、生のテキストではなく構造化された事実だけをメインエージェントへ返せます。他方、これは悪用され得ます——サブエージェントの出力が「自分たち発」だからと、生のツール結果より高い信頼レベルとして扱われると、プロンプトインジェクションの新たなベクトルが生まれます。マルチエージェントシステムでは、信頼レベルを使い分けることと、信頼の昇格に脆くなることとの間にトレードオフがあります。

エージェントのアイデンティティ。Claude Cowork のエージェントアイデンティティに対する答えは具体的です。クレデンシャルはホストのキーチェーンに残し、VM にはセッションごとに権限を絞ったトークンを渡し、そのトークンはユーザーのものとは独立に失効できます。一方で、クロスプラットフォームでのエージェントアイデンティティというより広い問いにも取り組み始めています。エージェントは自身の主体アイデンティティを持つべきか、それともユーザーの拡張として振る舞い、ユーザーの権限を継承すべきか。最終的な答えは、両者のブレンドになるかもしれません。

エージェントが能力を増すにつれ、攻撃面は絶えず移ろっていきます。私たちが見てきた失敗のパターンは、業界とラボの双方で繰り返されるでしょう。共有ベンチマークや開示の規範、共通のアイデンティティ標準、ベンダー横断のレッドチーミングに至るまで、エージェント固有のセキュリティ姿勢への集合的な投資が必要です。

本稿では封じ込めに焦点を当てましたが、これはエージェントのセキュリティ全体像の一部にすぎません。ガバナンス、可観測性、その他のスタックについては、[NIST の AI エージェントのアイデンティティと認可に関するプロジェクト](#)、オーストラリアの ACSC が CISA と英国の NCSC とともに主導した [エージェント型 AI 採用に関する 6 機関合同ガイダンス](#)、そして AI マネジメント標準である [ISO/IEC 42001](#) を参照してください。私たちの Glasswing イニシアチブはそのひとつの貢献ですが、この重要な課題についてはパートナーや競合の双方と協働できることを楽しみにしています。

まとめ

要約すると、私たちが繰り返し立ち戻る原則はいくつかあります。

まず環境層で封じ込めを設計し、その後でモデル層の挙動を導く。 最も多くを教えてくれた 2 つのインシデント——従業員フィッシングと、サードパーティが開示してくれた許可リストの問題——は、いずれもエグレス、つまり許可された経路を通じてデータが外へ出る事案でした。どちらの場合も、モデル層には手が出ません。検知すべき異常は何もなかったのです。確率的なものがすべて見落とすときに当たり受けるのは、決定的な境界です。

ユーザーが監督できる範囲に、隔離の強さを合わせる。 bash を読める開発者と、それができないナレッジワーカーは、同じ脅威モデルで動いているわけではありません。エージェントがやろうとしていることをユーザーが評価できるか——この問いが封じ込め戦略を決める手がかりになります。両方向のどちらに振っても——専門家にとっての過剰な摩擦、非専門家への過剰な信頼——それ自体が失敗です。

カスタムコンポーネントには慎重に。 実戦投入されてきたハイパーバイザー、syscall フィルタ、コンテナランタイムは、あなたが作るどんなものよりも長く敵対的な注目に耐えてきました。本稿で触れたすべての展開において、標準のプリミティブは持ちこたえ、その周囲に私たちが施した独自の作業が穴を露呈しました。

最後に、エージェントは新しいソフトウェアカテゴリーかもしれませんが、システムレベルの相互作用は新しくありません。エージェントは依然としてファイルを読み、ソケットを開き、プロセスを生成します——だからこそ成熟したツールによる封じ込めが、決定的に有効な防御となるのです。AI が発展するにつれ、展開のリスクとリターンのバランスは動き続けるでしょうが、爆発半径に硬い上限を置くことは、しばしばそのバランスを正しい方向へと押し戻してくれます。

謝辞

Written by Max McGuinness, Mikaela Grace, Jiri De Jonghe, Jake Eaton, and Abel Ribbink.

また、貢献いただいた Hanah Ho, Hasnain Lakhani, Pedram Navid, Molly Villagra, Maya Nielan, Akila Srinivasan, Sam Attard, Alfred Xing, Mohamad El Hajj, Gabby Curtis, David Dworken, Adam Jones, Amie Rotherham, Christian Ryan, Lucas Smedley, Brett Andrews 他の皆様に感謝します。

セキュリティチームとプロダクトエンジニアリングチーム、そして Claude 製品に対する脆弱性を報告してくださった個人や組織に特別な感謝を表します。

脚注

1. Claude Code auto mode はコマンド承認をモデルベースの分類器に委ね、摩擦を最小化 (無害なコマンドのうちブロックされるのはおよそ 0.4%) する代わりに、危険なコマンドの一部 (行き過ぎた行動のうち約 17%) を通してしまいます。そのため、サンドボックスを置き換えるものではなく、その内側に置く多層防御のひとつの層となります。